SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO2 - ASSESSMENT TOOL - 2

Reverse Engineering Task

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 1: Reverse Engineering Task**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO1 Statement	**Design effective test cases using black-box, white-box testing techniques and to control methods flow.**		
Student Name	**B.Thejaswini**	**Reg. No.**	**192412479**
Section/Batch	**Slot A /**	**Date**	**02-11-2026**

Code of Conduct:

I,B.Thejaswini, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B.Thejaswini

1. Introduction

A mobile payment gateway is a transaction-processing system that receives payment requests, communicates with banking or payment services, and returns API responses in JSON format. During processing, a transaction may move through several states such as Pending, Success, Reversal and Timeout. Correct state management is essential because the same payment must not be charged twice, incorrectly marked successful, or reversed after an invalid event.

The given scenario asks the tester to reverse engineer the underlying state machine from API responses, transition triggers and data constraints. The objective is to identify the legal and illegal transitions, reconstruct business rules, identify the implicit state-transition testing technique, and prepare a complete state-based test suite.

1.1 Problem Statement

The API exposes transaction states but the complete state-transition specification is not explicitly provided. Testers must infer the state machine from observed JSON responses, transaction events and constraints. The major risk is inconsistent state handling—for example, a successful transaction receiving another payment confirmation, a timeout being treated as success without verification, or a reversal being attempted from an invalid state.

1.2 Objectives

- Identify functional and non-functional requirements from the scenario.
- Reverse engineer the states Pending, Success, Reversal and Timeout.
- Identify triggers/events responsible for state changes.
- Determine legal and illegal state transitions.
- Identify the implicit state-transition testing technique.
- Design positive and negative state-transition test cases.
- Apply boundary and equivalence-class thinking to JSON fields.
- Construct a decision table for transaction outcomes.
- Identify potential defects and their business impact.
- Present a structured state-transition model for regression testing.

1.3 Assumptions

Because the question does not provide the actual JSON samples, this report uses a representative API contract for analysis. The assumed fields are transaction_id, state, amount, currency, timestamp, response_code, reference_id and reversal_id. In an actual laboratory submission, the observed JSON values should replace these illustrative values without changing the testing method.

2. Scenario Analysis and API Response Model

2.1 Representative JSON Responses

State	Representative response meaning	Typical trigger
Pending	Request accepted and transaction processing is still in progress.	Payment request accepted; external confirmation pending.
Success	Payment is completed successfully and has a valid transaction/reference identifier.	Valid confirmation from payment processor.
Reversal	Previously authorized/settled transaction has been reversed or cancelled.	Approved reversal request or compensating event.
Timeout	Transaction could not be confirmed within the permitted time.	Processing time exceeds configured timeout.

2.2 Representative JSON Contract

A typical successful response may contain: transaction_id, state='SUCCESS', amount, currency, timestamp, response_code='00', and reference_id. A pending response may contain a transaction identifier and state='PENDING'. A reversal response may include reversal_id or a reference to the original transaction. A timeout response should identify the transaction and indicate that confirmation was not received within the allowed period.

2.3 Functional Requirements

- FR1: The API shall accept a valid payment request.
- FR2: The API shall create a unique transaction identifier.
- FR3: The API shall return a valid initial/processing state.
- FR4: The transaction shall move from Pending to Success when valid confirmation is received.

- FR5: The transaction shall move from Pending to Timeout when the configured time limit expires without confirmation.
- FR6: A valid eligible transaction shall support reversal according to business policy.
- FR7: The API shall reject illegal state transitions.
- FR8: The API shall not allow duplicate successful settlement for the same transaction.
- FR9: The response shall contain mandatory fields appropriate to the current state.
- FR10: Every state-changing event shall be auditable using transaction identifiers and timestamps.

2.4 Non-Functional Requirements

- Security: transaction data and authorization information must be protected.
- Performance: state responses should be returned within the agreed API response time.
- Reliability: state changes must not be lost during network or service failures.
- Consistency: the same transaction should not simultaneously appear in incompatible final states.
- Availability: the gateway should recover safely after temporary service interruption.
- Traceability: every state change should be logged with an identifier and timestamp.

3. Reverse Engineering the State Machine

State-transition testing models the system as states connected by transitions. A transition occurs when an event or trigger changes the transaction from one state to another. The observed API response provides evidence about the current state, while the request/event and previous response help infer the transition.

3.1 Identified States

State	Meaning	Terminal?	Important constraints
PENDING	Transaction is accepted but final payment confirmation is not available.	No	Transaction ID must exist; amount/currency valid; processing window active.
SUCCESS	Payment is confirmed successfully.	Usually yes	Valid transaction ID, success code, reference ID and valid amount.
REVERSAL	A previously eligible transaction has been reversed.	Usually yes	Original transaction/reference must exist; reversal must be authorized and not duplicated.

TIMEOUT	Confirmation was not received before the allowed processing period.	Policy dependent	Transaction ID exists; timeout condition must be satisfied.

3.2 State Transition Triggers

Trigger	Description
T1: Payment Accepted	Gateway accepts a syntactically and semantically valid request.
T2: Success Confirmation	Processor/acquirer confirms successful payment.
T3: Timeout Expiry	Configured waiting period expires without valid confirmation.
T4: Reversal Request	Authorized request attempts to reverse an eligible transaction.
T5: Duplicate Request	Same transaction/event is received again.
T6: Invalid Event	Event is incompatible with the current transaction state.
T7: Late Success	Success confirmation arrives after timeout.
T8: Retry/Status Check	Client asks for current status or retries safely.

3.3 Implicit State Transition Testing Technique

The implicit technique used is State Transition Testing, specifically model-based state transition coverage inferred from observed behavior. The tester identifies states, events, transitions, valid transitions and invalid transitions, then creates tests to cover them. Because the original state model is not directly supplied, this is a reverse-engineered or inferred state model.

3.4 State Transition Principle

The important testing question is not only 'Is the JSON response correct?' but also 'Is this JSON response legal for the previous state and event that occurred?' This distinction detects sequence-related defects that ordinary single-response testing can miss.

4. Structured State-Transition Diagram

The following textual diagram can be directly converted into a hand-drawn or graphical state-transition diagram for submission.

START → PENDING

PENDING -- valid success confirmation --> SUCCESS

PENDING -- timeout expiry --> TIMEOUT

PENDING -- authorized eligible reversal (if policy permits) --> REVERSAL

SUCCESS -- authorized reversal --> REVERSAL

SUCCESS -- duplicate success/settlement --> REJECT / remain SUCCESS

TIMEOUT -- late success --> RECONCILIATION / policy-defined outcome

TIMEOUT -- invalid reversal --> REJECT / remain TIMEOUT

REVERSAL -- duplicate reversal --> REJECT / remain REVERSAL

Any state -- invalid event --> REJECT + audit log; state remains unchanged

4.1 Recommended Diagram Structure

Current state	Event/trigger	Guard/condition	Next state	Invalid case
PENDING	Success confirmation	Valid confirmation	SUCCESS	Invalid/duplicate confirmation
PENDING	Timeout	Time > configured limit	TIMEOUT	Timeout before limit
PENDING	Reversal	Policy allows reversal	REVERSAL	Unauthorized/ineligible reversal
SUCCESS	Reversal	Original transaction eligible	REVERSAL	Duplicate/unauthorized reversal
SUCCESS	Success	Duplicate event	SUCCESS / reject event	Second settlement
TIMEOUT	Late success	Confirmation after timeout	Policy/reconciliation	Blindly mark success
TIMEOUT	Reversal	Eligibility checked	REVERSAL or reject	Invalid reversal
REVERSAL	Reversal	Duplicate request	REVERSAL / reject	Second reversal
ANY	Invalid event	Event not allowed	No state change	Reject + log

4.2 State Invariants

- A transaction should have exactly one current state at a given logical point.
- A final state should not silently move to another final state without an explicitly supported business event.
- Every transition must be attributable to a transaction ID and event.
- Duplicate events must be idempotent or safely rejected.
- Mandatory JSON fields must match the current state.

- Amount and currency must remain consistent across the transaction lifecycle unless an explicitly defined adjustment is permitted.

5. Legal and Illegal Transition Analysis

5.1 Legal Transitions

From	Event	To	Reason
PENDING	Success confirmation	SUCCESS	Normal successful completion.
PENDING	Timeout expiry	TIMEOUT	No confirmation within allowed window.
PENDING	Eligible reversal	REVERSAL	Allowed compensating action under policy.
SUCCESS	Authorized reversal	REVERSAL	Successful transaction is reversed through valid process.

5.2 Illegal or Restricted Transitions

From	Event	Expected result	Reason
REVERSAL	Success confirmation	Reject / no state change	Reversal should not silently become success.
REVERSAL	Second reversal	Reject / idempotent response	Prevents duplicate reversal.
SUCCESS	Second success settlement	Reject / idempotent response	Prevents double charging.
TIMEOUT	Immediate success without reconciliation	Reject or controlled reconciliation	Late confirmation needs defined handling.
TIMEOUT	Invalid reversal	Reject	Eligibility must be verified.
ANY	Malformed event	Reject	Invalid event cannot change state.
ANY	Unknown state value	Reject	Unknown state is unsafe for financial processing.

5.3 Why Illegal Transitions Matter

Testing only legal paths is insufficient. Many severe payment defects occur when an event arrives in an unexpected sequence. Illegal-transition tests verify that the gateway protects transaction integrity when clients retry, networks duplicate messages, processors respond late, or users perform actions in the wrong order.

5.4 Idempotency

Payment APIs commonly require idempotent behavior. If the same payment request is submitted more than once with the same idempotency key or transaction reference, the gateway should not create multiple successful charges. The test suite must therefore include repeated requests and repeated state events.

6. Equivalence Classes and Boundary Conditions

6.1 JSON Input Equivalence Classes

Field	Valid class	Invalid class	Example
transaction_id	Non-empty unique identifier	Null, empty, duplicate where uniqueness required	TXN1001
amount	Positive value within permitted range	0, negative, above maximum	₹500
currency	Supported currency code	Unsupported/malformed code	INR
state	Known allowed state	Unknown/null/malformed state	PENDING
response_code	Valid defined code	Unknown/malformed code	00
timestamp	Valid timestamp within acceptable format/window	Missing, malformed, impossible date	ISO timestamp
reference_id	Required for successful confirmation	Missing/duplicate where mandatory	REF1001
reversal_id	Unique reversal identifier when reversal occurs	Duplicate/null when mandatory	REV1001

6.2 Boundary Values

Parameter	Boundary tests
Amount	Minimum valid, just below minimum, just above minimum, maximum valid, just above maximum
Timeout	One unit before timeout, exact timeout, one unit after timeout
Transaction ID length	Minimum allowed length, below minimum, maximum allowed length, above maximum

Retry count	0 retries, maximum allowed retries, maximum+1
Timestamp	Just before allowed window, exact boundary, just after boundary
Payload size	Minimum valid payload, maximum permitted payload, maximum+1

6.3 Timeout Boundary Example

If the configured timeout is 30 seconds, test at 29 seconds, exactly 30 seconds and 31 seconds. At 29 seconds, the transaction should remain eligible for normal confirmation. At exactly 30 seconds, the implementation must follow the documented timeout rule. At 31 seconds, the timeout transition should already have occurred unless the system uses a different defined tolerance.

6.4 Performance and Reliability Constraints

- Repeated status requests should not create a new payment.
- Duplicate callback events should not create duplicate settlement.
- Late callbacks should follow a documented reconciliation rule.
- Concurrent success and timeout events must have deterministic processing.
- API retries must preserve transaction identity and state integrity.

7. Decision Table for Transaction State Handling

The decision table reconstructs the business rules by combining current state and incoming event conditions.

Rule	Current state	Event	Condition	Expected action
R1	PENDING	Success	Valid confirmation	Move to SUCCESS
R2	PENDING	Timeout	Timeout reached	Move to TIMEOUT
R3	PENDING	Reversal	Reversal permitted	Move to REVERSAL
R4	PENDING	Reversal	Not permitted	Reject; remain PENDING
R5	SUCCESS	Reversal	Authorized and eligible	Move to REVERSAL
R6	SUCCESS	Success	Duplicate event	Reject/idempotent; remain SUCCESS
R7	TIMEOUT	Success	Late confirmation	Apply reconciliation policy
R8	TIMEOUT	Reversal	Eligible	Move to REVERSAL if policy allows
R9	REVERSAL	Reversal	Duplicate	Reject/idempotent; remain REVERSAL

R10	ANY	Unknown/malformed event	Invalid	Reject and audit; no state change

7.1 Decision Table Testing

Each rule in the table becomes at least one test condition. This provides a direct link between business rules and test cases. If a rule cannot be tested because no API mechanism exists to trigger it, the gap should be documented rather than silently omitted.

7.2 Functional Requirements Traceability

Requirement	**Related states/events**	**Test coverage**
FR4 Success transition	PENDING + Success	ST01–ST04
FR5 Timeout transition	PENDING + Timeout	ST05–ST07
FR6 Reversal	SUCCESS/PENDING + Reversal	ST08–ST12
FR7 Illegal transition protection	All states + invalid events	ST13–ST20
FR8 Duplicate protection	SUCCESS/REVERSAL + duplicate	ST21–ST24
FR9 JSON constraints	All responses	ST25–ST30
FR10 Auditability	Every transition	ST31–ST33

8. Comprehensive State-Based Test Suite

ID	Initial state	Event/input	Expected result	Type
ST01	PENDING	Valid success confirmation	SUCCESS; valid response code/reference	Positive
ST02	PENDING	Success confirmation with valid amount/currency	SUCCESS	Positive
ST03	PENDING	Malformed success callback	Reject; remain PENDING	Negative
ST04	PENDING	Duplicate success callback	No duplicate settlement; idempotent/reject	Negative
ST05	PENDING	Timeout at exact configured limit	TIMEOUT according to policy	Boundary
ST06	PENDING	Timeout before configured limit	Remain PENDING	Boundary
ST07	PENDING	Timeout after configured limit	TIMEOUT	Boundary
ST08	PENDING	Authorized reversal if allowed	REVERSAL	Positive
ST09	PENDING	Unauthorized reversal	Reject; remain PENDING	Negative
ST10	SUCCESS	Authorized reversal	REVERSAL	Positive
ST11	SUCCESS	Unauthorized reversal	Reject; remain SUCCESS	Negative
ST12	SUCCESS	Duplicate reversal	Reject/idempotent	Negative
ST13	SUCCESS	Second success event	Reject/idempotent; remain SUCCESS	Negative
ST14	TIMEOUT	Late success	Controlled reconciliation outcome	Negative/Recovery
ST15	TIMEOUT	Invalid reversal	Reject; remain TIMEOUT	Negative
ST16	TIMEOUT	Eligible reversal	Policy-defined reversal	Positive
ST17	REVERSAL	Second reversal	Reject/idempotent	Negative
ST18	REVERSAL	Success event	Reject; remain REVERSAL	Negative
ST19	ANY	Unknown state	Reject response	Negative
ST20	ANY	Malformed JSON	Reject with validation error	Negative
ST21	PENDING	Missing transaction_id	Reject	Negative
ST22	PENDING	Zero amount	Reject	Boundary/Negative
ST23	PENDING	Negative amount	Reject	Negative
ST24	PENDING	Unsupported currency	Reject	Negative
ST25	SUCCESS	Missing reference_id	Reject/flag inconsistency	Negative
ST26	SUCCESS	Duplicate transaction_id	Reject/idempotent	Negative

ST27	PENDING	Same request repeated	Same transaction outcome; no double charge	Positive/Idempotency
ST28	PENDING	Concurrent success and timeout	Deterministic single outcome	Concurrency
ST29	PENDING	Retry before timeout	Remain PENDING or safe status response	Positive
ST30	PENDING	Retry after timeout	Return TIMEOUT/reconciliation status	Negative/Recovery

9. Detailed Test Cases and Expected JSON Validation

9.1 Success Response

For a successful transaction, verify state='SUCCESS', a valid response code, a non-empty transaction/reference identifier, correct amount and currency, and a valid timestamp. The response must correspond to the same transaction that was submitted.

9.2 Pending Response

For a pending transaction, verify state='PENDING', a valid transaction identifier and an appropriate processing status. The system must not expose a successful settlement amount as final unless success has been confirmed.

9.3 Timeout Response

For timeout, verify state='TIMEOUT', transaction identity, timestamp and defined timeout/error code. The system should not silently return SUCCESS merely because a client retries after the timeout.

9.4 Reversal Response

For reversal, verify state='REVERSAL', the original transaction reference, reversal identifier where required, correct amount and an appropriate reversal code. Duplicate reversal requests must not cause a second reversal.

9.5 Sample Detailed Cases

ID	Steps	Expected
D01	Create valid payment → receive PENDING → send valid success callback	State becomes SUCCESS exactly once; reference ID exists.

D02	Create valid payment → wait until just before timeout → query status	State remains PENDING if no timeout event has occurred.
D03	Create valid payment → reach exact timeout threshold	System follows documented exact-boundary timeout rule.
D04	Create valid payment → exceed timeout → query	State is TIMEOUT or documented reconciliation state.
D05	Complete payment → send authorized reversal	State becomes REVERSAL and original transaction is linked.
D06	Complete payment → send reversal twice	First reversal succeeds; second is rejected/idempotent.
D07	Timeout transaction → send late success	System follows explicit reconciliation rule; no uncontrolled double settlement.
D08	Send malformed JSON	Request is rejected without changing transaction state.

10. Potential Defects and Their Impact

Defect	Example	Impact	Severity
Wrong transition	PENDING directly becomes SUCCESS without valid confirmation	Unauthorized payment completion	Critical
Duplicate settlement	Repeated success callback creates two charges	Financial loss/customer dispute	Critical
Incorrect timeout	Timeout occurs too early/late	Incorrect customer status and reconciliation	High
Late success mishandled	TIMEOUT becomes SUCCESS without reconciliation	Inconsistent financial records	Critical
Duplicate reversal	Same payment reversed twice	Incorrect ledger balance	Critical
Missing field validation	SUCCESS response has no reference ID	Audit/reconciliation failure	High
Unknown state accepted	API accepts unsupported state	State-machine corruption	Critical
Race condition	Success and timeout processed inconsistently	Non-deterministic payment outcome	Critical
Amount mismatch	Callback amount differs from original	Wrong amount settled	Critical
Currency mismatch	Original INR, callback different currency	Incorrect financial calculation	High

10.1 Defect Life-Cycle

- Detect the unexpected state or response.
- Record initial state, triggering event, request JSON and response JSON.
- Compare actual transition with the decision table.
- Assign severity based on financial and customer impact.
- Reproduce using the same transaction/event sequence.

- Fix the transition logic or validation.
- Re-run the original test and the full state-transition regression suite.

10.2 Evidence to Capture

- Transaction ID and idempotency key.
- Initial state and previous API response.
- Exact event/request JSON.
- Response JSON and HTTP status.
- Timestamp and elapsed time.
- Expected state versus actual state.
- Relevant server/audit logs.
- Defect ID and environment information.

11. State Transition Coverage and Regression Strategy

11.1 Coverage Types

Coverage	Objective
State coverage	Visit every defined state at least once.
Valid transition coverage	Execute every legal transition.
Invalid transition coverage	Attempt every important illegal transition.
Event coverage	Exercise each event/trigger.
Transition-pair coverage	Test sequences of two connected transitions.
Boundary coverage	Test timing and data boundaries around transitions.
Data constraint coverage	Test valid/invalid JSON values.
Concurrency coverage	Test simultaneous or racing events.

11.2 Transition Coverage Matrix

From \ Event	Success	Timeout	Reversal	Duplicate/Invalid
PENDING	SUCCESS	TIMEOUT	REVERSAL or reject	Reject
SUCCESS	Reject/idempotent	Reject/retain	REVERSAL	Reject
TIMEOUT	Reconciliation policy	Retain	REVERSAL or reject	Reject
REVERSAL	Reject	Retain	Reject/idempotent	Reject

11.3 Regression Tests

- Every production fix involving state logic must trigger the complete transition regression suite.
- Always repeat duplicate payment and duplicate reversal tests.
- Always repeat timeout boundary tests.
- Always test late callback handling.
- Always verify amount/currency consistency.
- Always test idempotency after changes to request processing.
- Run API schema validation after changes to JSON fields.

11.4 State Transition Coverage Metrics

A useful coverage measure is Transition Coverage = executed valid transitions / total required valid transitions × 100. Invalid Transition Coverage can similarly measure the number of specified illegal transitions attempted and correctly rejected. State Coverage = visited states / total defined states × 100. These metrics make the completeness of the test suite measurable.

12. Analysis of the Implicit Testing Technique

The observed scenario implicitly uses State Transition Testing because transaction behavior depends on the previous state and the event that occurs. A response cannot be evaluated correctly in isolation. For example, a SUCCESS response after a valid confirmation from PENDING is legal, but the same SUCCESS event after REVERSAL is generally illegal.

12.1 Why State Transition Testing is Appropriate

- Payment systems are inherently sequence-dependent.
- Failures often appear only after a particular event order.
- Retries and callbacks can produce duplicate events.
- Timeout and late responses create asynchronous transitions.
- Reversal depends on the previous transaction state.
- State-based tests expose defects that ordinary input/output tests can miss.

12.2 Comparison with Other Black-Box Techniques

Technique	Main use	Relevance here

ECP	Reduce input domain into valid/invalid classes	Useful for JSON fields and amounts.
BVA	Test values around boundaries	Useful for timeout and amount limits.
Decision Table	Test combinations of business conditions	Useful for state + event rules.
State Transition	Test states, events and sequences	Primary technique for this scenario.
Exploratory	Discover unexpected behavior	Useful complementary technique.

12.3 Combined Approach

The strongest solution is not to use state transition testing alone. State Transition Testing should form the core model, while ECP tests JSON data classes, BVA tests timeout and numeric limits, and Decision Table Testing verifies reconstructed business rules. Exploratory testing can then probe unusual sequences and integration failures.

13. Complete Test Execution Checklist

- Requirement review completed.
- All four observed states documented.
- All state triggers identified.
- Legal transitions documented.
- Illegal transitions documented.
- Decision table prepared.
- State-transition diagram prepared.
- Valid and invalid JSON equivalence classes identified.
- Timeout boundaries tested.
- Amount boundaries tested.
- Duplicate request behavior tested.
- Duplicate callback behavior tested.
- Late success behavior tested.
- Reversal eligibility tested.
- Concurrent event behavior tested.
- Mandatory response fields validated.
- Transaction ID consistency checked.

- Amount and currency consistency checked.
- Audit logging verified.
- Regression suite updated after defect fixes.

14. Expected Results

The completed suite should demonstrate that every legal state transition occurs under the correct trigger and every important illegal transition is rejected or handled through a documented recovery rule. No transaction should be double-settled, double-reversed, or moved to an incompatible state without an authorized business event.

15. Conclusion

The mobile payment gateway can be reverse engineered as a state-driven system consisting of Pending, Success, Reversal and Timeout states. The implicit testing technique is State Transition Testing, supported by transition coverage and invalid-transition testing. The reconstructed model identifies the events that cause state changes and the conditions under which those changes are legal.

A comprehensive test suite must cover both positive and negative paths. Positive tests confirm that valid payment, timeout and reversal operations reach the correct states. Negative tests verify that duplicate, malformed, unauthorized, late or out-of-sequence events do not corrupt the transaction state. ECP and BVA strengthen the suite by testing data constraints and transition boundaries such as timeout thresholds. Decision tables provide traceability from business rules to test cases.

RUBRICS FOR CALCULATION

Criteria	**Weightage (%)**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**	**Marks**
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality.	Shows good understanding with minor gaps.	Partial understanding; misses key aspects.	Lacks understanding of the system/product.	
Decomposition	25%	Effectively breaks down the system into components with detailed analysis.	Decomposes system with minor gaps in analysis.	Limited or incomplete decomposition.	Unable to analyze or break down the system.	
Identification of Components	20%	Accurately identifies all components and explains their functions clearly.	Identifies most components with minor errors.	Identifies few components; explanations are weak.	Unable to identify components or functions.	

Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic.	Reconstruction is mostly correct with minor issues.	Partial reconstruction; lacks clarity.	Unable to reconstruct or model system.	
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation.	Documentation is adequate with minor issues.	Poorly organized or incomplete documentation.	No proper documentation or presentation.	